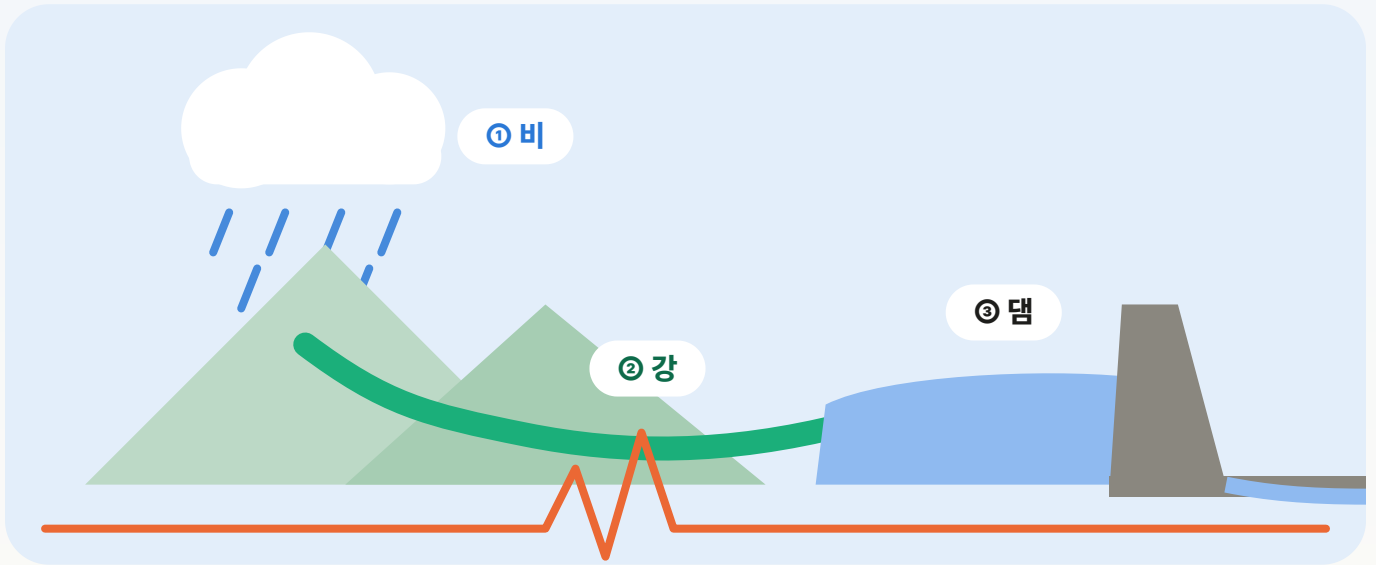


WaterPulse

한눈에 보기

비가 오면, 몇 시간 뒤에 댐으로 물이 몰려올까?



1

과제가 원하는 것

무엇을, 언제까지, 무엇으로 평가받나 · 2쪽

2

WaterPulse는 무엇인가

주제 설명과 보완 제안 4가지 · 4쪽

3

API와 데이터 수집

API 5개 신청 완료, 받는 데 걸리는 날, 수집기도 과제인가 · 8쪽

4

하둡 도구와 전체 흐름

도구별 뜻·장단점·우리 선택, 도구가 실제로 하는 일 · 11쪽

5

분석과 기대 결과

추세·예측·비교 분석을 어떻게 하나, 한계와 예외, 기대 결과 · 18쪽

✓ 10월 5일까지 끝난 것

주제 확정 (팀원 제안 WaterPulse)

API 5개 신청 · 승인

실제 호출로 데이터 확인

데이터 함정 6가지 발견

다 받는 데 걸리는 날 계산

한 문장으로 말하면

공공 데이터를 잔뜩 모아 하둡 창고에 정리하고,
그걸로 쓸모 있는 분석을 해서, 웹 화면으로 보여 주세요.



고르기

어떤 데이터?

공공 API



모으기

실어 오기

수집기 ·
Flume ·
Sqoop



창고에 정리

칸 나눠 넣기

HDFS ·
Hive ·
Impala



분석

숨은 답 찾기

Spark



보여 주기

한눈에

웹
대시보드



아래 두 가지는 똑같이 중요하게 평가합니다

과제 설명서 원문: "빅데이터 관리와 분석은 동등하게 중요하게 다뤄진다"

요구 ①

빅데이터 관리

엑셀이나 일반 DB가 아니라 하둡
도구로 답아야 합니다.

채점 포인트: 왜 그 도구를 골랐고,
얼마나 효율적인가

요구 ②

의미 있는 분석

평균·합계 표로는 부족합니다.

× "월별 평균
유입량 표"

✓ 예측 · 비교 ·
추세

💡 과제 설명서의 예시 4개(트위터 트렌드, 교통카드, 쇼핑몰 주문, 대기질)도 모두 위 다섯 단계 구조입니다.

언제, 무엇을 내고, 무엇으로 평가받나

오늘

10%

20%

10월 5일

주제 확정 · API 신청 완료

10월 19일(월) 14시

중간발표 · 아이디어 제안 · 영상 약 10분

동료평가 10/21까지

12월 7일(월) 14시

최종발표 · 구현 결과 · 영상 약 15분
동료평가 12/9까지

🎤 중간발표에 꼭 넣을 것

💡 **아이디어** — 어떤 데이터로, 무엇을 할지

🕒 **전체 구조 그림** — 화면·하둡·DB·웹 서버의 관계

🔧 **구성요소 설명** — 각 부분이 하는 일

📝 **중간 결과** — 샘플 데이터를 실제로 하둡에 넣고 돌려 본 것

📅 **프로젝트 계획** — 12월까지 일정

🏁 최종발표에 꼭 넣을 것

✅ **최종 결과와 구성요소 상세 설명**

🐛 **문제를 찾고 고친 과정 (디버깅)**

📦 **소스코드 + 매뉴얼 압축 파일**

💻 **매뉴얼대로 하면 수업 실습 VM에서 똑같이 돌아가야 함**

🎥 **영상에 발표하는 팀원 모두의 얼굴이 나와야 함**

📝 채점 기준

중간

🎨 창의성

🔧 학기 안에 가능한가

🧩 완성도 (예외 상황까지 생각했나)

📅 계획

🎤 발표

최종

🎯 설계대로 됐나

🧩 완성도

🐛 문제 분석·디버깅

📄 데이터 관리의 깊이

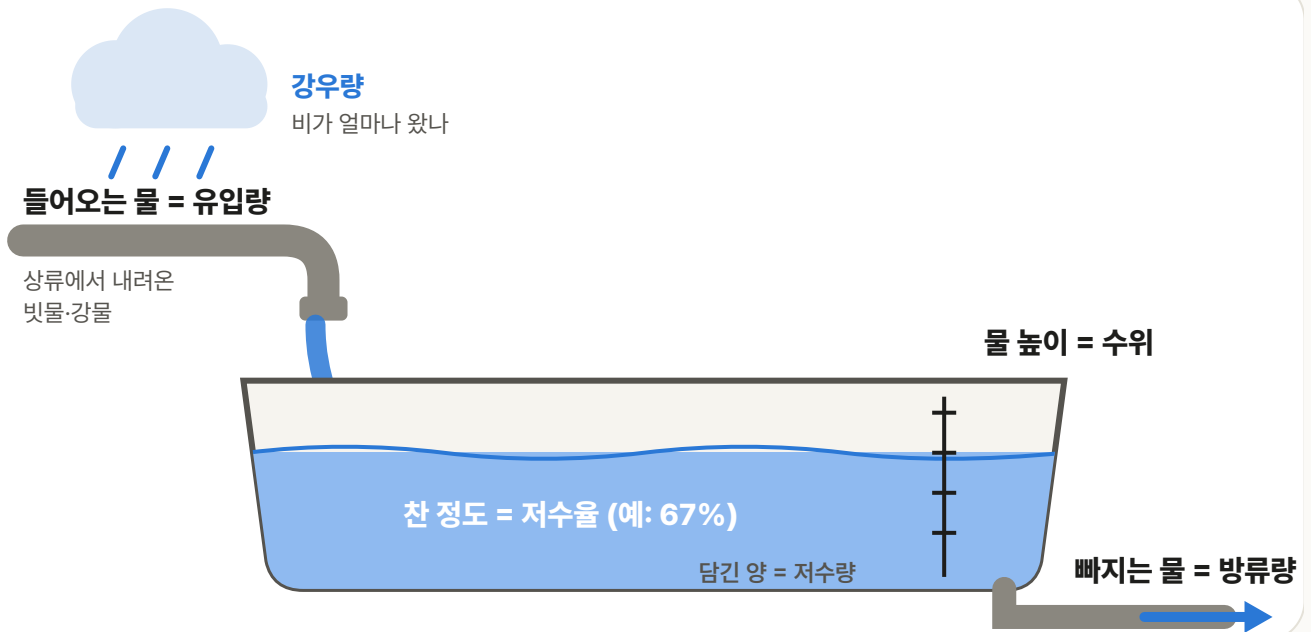
🔬 분석의 깊이 (한계·예외 포함)

🎤 발표

⚠️ 늦게 내면 받지 않습니다. 제출은 녹화 영상 + 발표 슬라이드 + 보충 자료(소스코드 등)를 Google Drive에.

댐을 커다란 욕조라고 생각해 보세요

팀원이 제안해 10월 5일 확정된 주제입니다.



📍 실제 숫자로 보면 — 소양강댐, 2026년 10월 1일 01시 (API로 받은 값)

물 높이(수위)	들어오는 물(유입량)	빠지는 물(방류량)	찬 정도(저수율)
185.07 m	28.4 m³/s	0 m³/s	67.1 %

🤔 비가 오면 몇 시간 뒤에 욕조(댐)가 차오를까?
그리고 그걸 미리 맞힐 수 있을까?



대상

한국수자원공사(K-water)가 관리하는 댐·보 59곳



데이터 단위

10분 · 1시간 · 1일마다
측정한 값

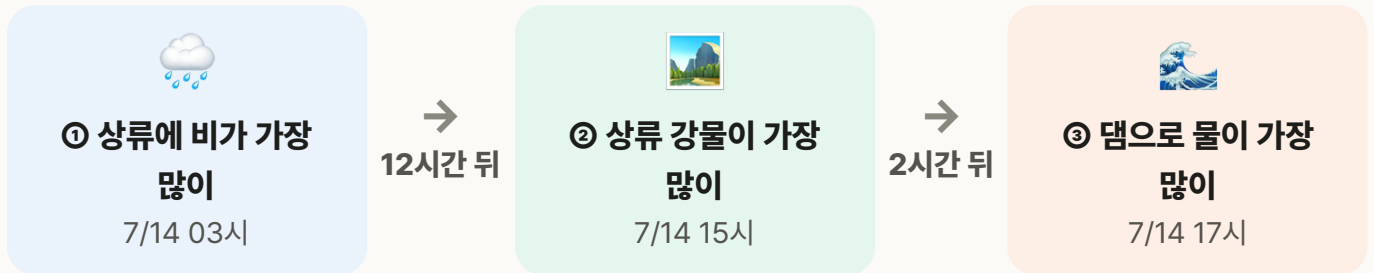


쌓인 기간

댐은 2006년부터, 상류
관측소는 1996년부터

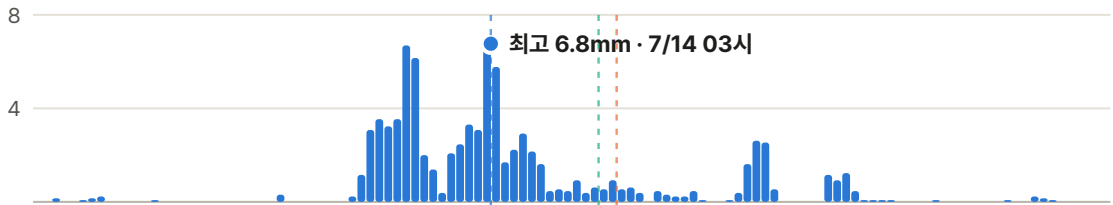
실제로 이런 일이 일어납니다

2023년 7월 장마 때 소양강댐 주변. API로 직접 받은 실제 값입니다.



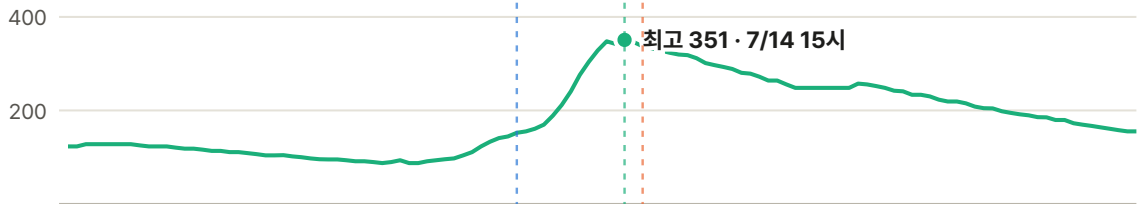
상류에 내린 비

우량관측소 13곳 평균 · mm/시간



상류 강물

인제군 원대교 수위관측소 유량 · m³/s



댐으로 들어온 물

소양강댐 유입량 · m³/s



물은 **비** → **강** → **댐** 순서로 내려옵니다.
이 "시간 차"를 재고, 미리 맞히는 것이 WaterPulse입니다.

⚠ 비 한 번만 본 그림입니다. "소양강댐은 항상 12시간"이라는 뜻이 아니라, 이런 시간 차를 수천 번의 비에서 재보자는 것이 분석입니다.

WaterPulse가 하려는 4가지



① 시간 차 재기

비가 온 뒤 댐마다 몇 시간
만에 물이 몰려오나? 댐끼리
비교하기



② 미리 맞히기

3·6·12시간 뒤 댐에 들어올
물의 양 예측하기



③ 이상한 순간 찾기

예상보다 훨씬 많이(적게)
들어오면 표시. “사고”가
아니라 평소와 다른 변화라고
부르기



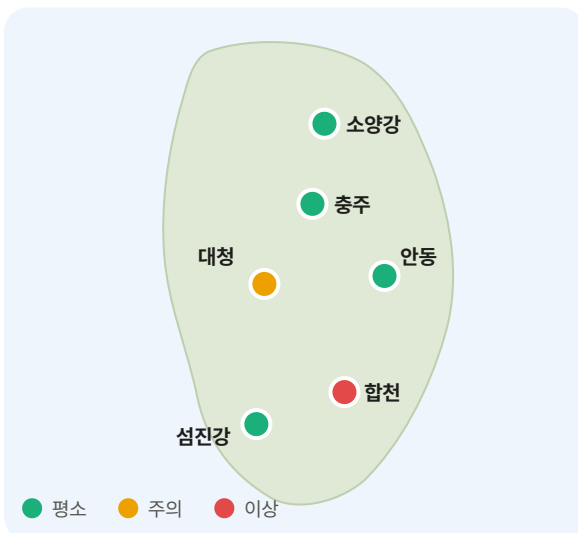
④ 비슷한 댐끼리 묶기

빨리 반응하는 댐 · 천천히
반응하는 댐 · 계절을 많이
타는 댐

완성되면 이런 화면이 됩니다

예시 · 숫자는 가상

WaterPulse · 전국 댐 현황



소양강댐

평소

지금 수위

185.1m

지금 유입량

420m³/s

저수율

67%

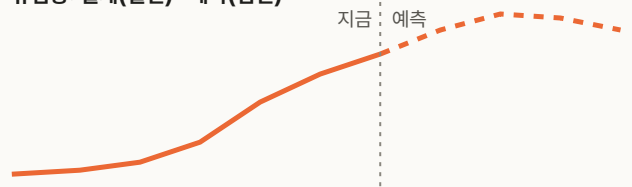
3시간 뒤 예측

480m³/s

6시간 뒤 예측

610m³/s

유입량: 실제(실선) · 예측(점선)



이렇게 보완하면 점수가 더 잘 나옵니다

주제는 그대로 두고, 채점 기준에 맞춰 네 군데만 손봅니다.

1

지금 제안서 · 창의성

교수님 예시와 뼈대가 같음

과제 설명서 예시 4(대기질: API로 모으고, 날짜·지역으로 나누고, 날씨와 비교하고, 지도로 보여 주기)와 구조가 거의 같습니다.

→

이렇게

“비 → 강 → 댐” 흐름을 간판으로

관측소 API는 댐마다 상류 관측소 목록을 줍니다. 5쪽 그림처럼 물이 내려오는 길을 따라가는 것이 다른 팀과 다른 점입니다.

2

지금 제안서 · 데이터 관리

Flume·Sqoop 쓸 자리가 없음

데이터가 전부 API라서 수업에서 배운 수집 도구가 저절로 들어가지 않습니다. “효율적”이라는 말에 근거도 없습니다.

→

이렇게

도구는 일부러 넣고, 효율은 숫자로

받은 파일은 **Flume**으로 창고에, 댐 좌표표는 MySQL에 두고 **Sqoop**으로 가져오기, 결과는 Sqoop으로 웹 DB에. 압축 형식(Parquet) 전후 용량과 칸 나누기(파티션) 전후 조회 시간을 재서 보여 주기.

3

지금 제안서 · 분석

4가지를 다 하면 다 알아짐

남은 기간은 약 두 달. 네 개를 같은 무게로 하면 어느 것도 깊어지지 않습니다.

→

이렇게

순서를 정해서 깊게

① 시간 차 + 댐 크기·유역 넓이로 이유 설명 → ② 예측. 과제가 첫째로 꼽는 추세도 하나 추가 (18쪽). “지금 양이 그대로 간다”고 짚는 것보다 나은지, 큰비 때만 따로 채점 → ③ 이상 찾기는 예측의 덤 → ④ 묶기는 시간이 남으면.

4

지금 제안서 · 완성도

예외 상황 이야기가 없음

채점 기준에 “예외적인 경우를 고려하는가?”가 있는데 제안서에는 데이터 문제 이야기가 없습니다.

→

이렇게

찾아낸 함정 6가지를 발표에

오늘 API를 직접 불러 보며 찾은 데이터 함정 6가지(17쪽)를 “미리 발견해서 처리한 예외”로 한 장 만들기.

한 줄 요약: 무엇을(상류 흐름)은 더 날카롭게, 어떻게(관리 도구·측정값)는 더 구체적으로, 얼마나(분석 개수)는 더 적게.

쓸 API 5개, 모두 신청 끝났습니다



10월 5일 신청 → 바로 자동 승인 → 실제로 불러서 데이터가 오는 것까지 확인

공공데이터포털 개인 인증키(팀 .env에 있는 키)로 신청. 기능마다 하루 10,000번까지 부를 수 있습니다.



수문 운영 정보

필수 · 주인공

✓ 승인·확인

한국수자원공사 · 15099110

“댐의 상태” — 댐·보 59곳의 물 높이(수위), 비(강우량), 들어온 물(유입량), 내보낸 물(방류량), 담긴 양(저수량), 찬 정도(저수율)

10분 · 1시간 · 1일

2006년부터

한 번에 약 400줄씩



댐코드 조회

필수

✓ 승인·확인

한국수자원공사 · 15099105

“댐 이름표” — 댐 59개의 번호(예: 소양강 1012110). 다른 API를 부를 때 꼭 필요한 주소입니다.

하루 100번이면 충분



우량수위 관측소 운영 정보

핵심 확장

✓ 승인·확인

한국수자원공사 · 15099115

“댐 위쪽의 비와 강” — 댐 번호를 넣으면 그 댐 상류 관측소를 알려 줌. 비 관측소 187곳(시간 강우·누적 강우), 강 관측소 244곳(강 수위·유량)

10분 · 1시간

1996년부터

한 번에 500줄

“비 → 강 → 댐”을 가능하게 하는 API



수문 제원 현황

분석용

✓ 승인·확인

한국수자원공사 · 15099107

“댐 신체검사표” — 높이, 길이, 유역 넓이, 총 저수용량, 계획홍수위, 준공일. 댐마다 반응이 다른 이유를 설명할 때 씬

⚠ 위치(좌표)는 없음 → 지도용 좌표표는 직접 만들기



기상청 지상(ASOS) 시간자료

선택

✓ 승인·확인

기상청 · 15057210

“날씨” — 기온, 강수량, 습도, 기압, 일조. 예측에 날씨를 더 넣고 싶을 때

1시간

한 번에 받을 줄 수는 확인 필요



예비: WAMIS 국가수자원관리종합정보시스템

비상용

환경부

인증키 없이 바로 됩니다. 소양강댐 값이 K-water와 똑같았습니다(댐 49곳). K-water API가 막힐 때 대신 씁니다.

데이터를 다 받는 데 얼마나 걸리나



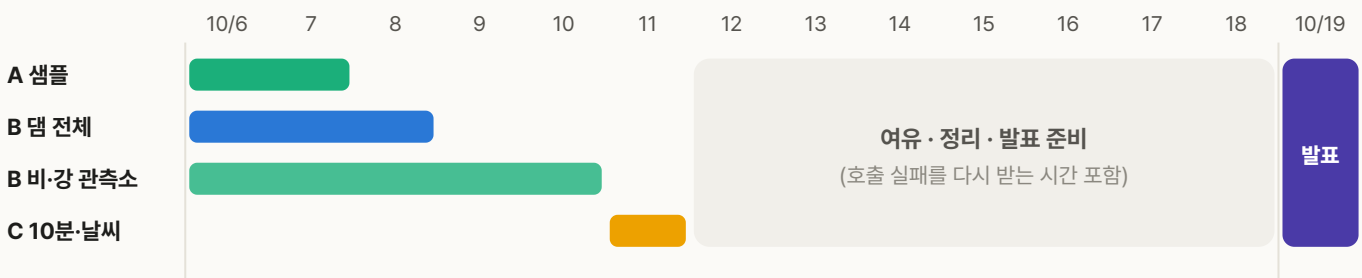
하루에 기능별 10,000번까지만
댐 자료·비 관측소·강 관측소는 기능이
달라서 동시에 받을 수 있습니다.



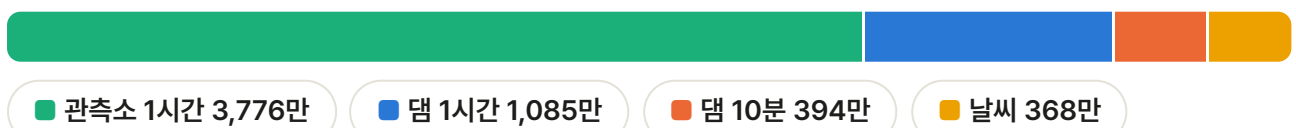
한 번에 댐 약 400줄, 관측소 500줄
더 달라고 하면 오류 없이 빈 답이 옵니다.
관측소는 999줄이 1쪽은 되고 6쪽은 빈
답이었습니다(실데이터 시뮬레이션에서
확인).

무엇을	몇 번 불러야 하나 (1시간 단위 자료)	걸리는 날
A 중간발표용 샘플 주요 댐 15곳 × 2021~2025년 + 그 댐의 상류 관측소 285곳	댐 1,650번 · 비 12,960번 · 강 12,690번 관측소 기능이 하루 한도를 조금 넘어 이틀	2일
B 최종용 전체 댐 59곳 × 2006~2026년 + 관측소 431곳 × 2016~2025년	댐 27,258번 (3일) · 비 33,660번 (4일) · 강 43,920번 (5일) 동시에 돌리면 가장 오래 걸리는 강 관측소 기준	약 5일
C 댐 10분 자료(댐 15곳×5년), 날씨 (20곳)	10분 자료 9,900번 · 날씨 7,560번	1일

17 오늘(10/5) 기준으로 이렇게 됩니다



다 모으면 최대 약 5,623만 줄



⚠ 최대치 계산입니다. 지점마다 자료가 있는 기간이 달라 실제는 더 적습니다. 속도는 문제가 아닙니다. 실데이터 시뮬레이션에서 관측소 72곳 × 6년을 받는 데 24분이 걸렸습니다. 병목은 “기능별 하루 1만 번” 한도(신청 화면 표시값)입니다.

수집기도 과제에 들어가나요?

수집기 = 공공 API를 불러 데이터를 파일로 받아 오는 우리 프로그램(Python)

네. API에서 받아 오는 것부터가 과제의 시작입니다.

다만 채점의 중심은 그다음, 하둡에 넣고 관리하고 분석하는 부분입니다.

과제 설명서에 이렇게 적혀 있습니다

① 과제 전체 흐름

"data sources → ingestion → big data management → analysis → Web visualization"

두 번째 칸 수집(**ingestion**)이 우리 몫입니다.

② 가장 짧은 예시 4 (대기질)

"...from public open APIs using a periodic collector script"

주기적으로 도는 수집 스크립트가 예시에 그대로 있습니다.

③ 중간발표

"Present how to collect, store, and analyze the data sets"

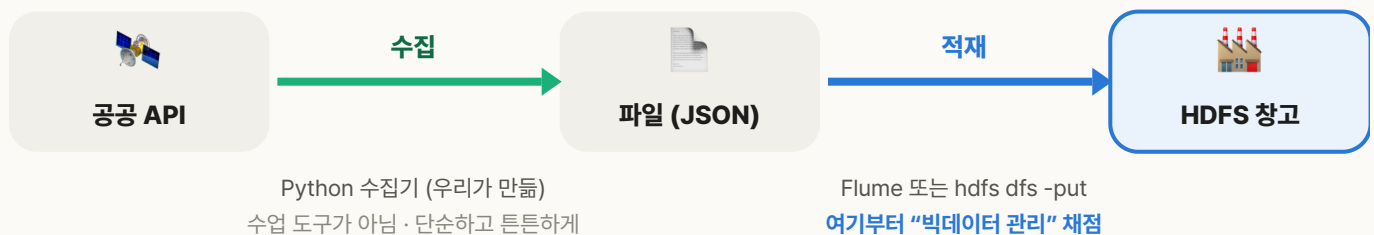
어떻게 모을지도 발표 내용입니다.

④ 최종 제출물

"...all the source codes, how to obtain the data, and a manual..."

데이터 얻는 법을 코드와 매뉴얼로 내야 합니다.

✂ "수집"과 "적재"를 나눠 생각하면 쉽습니다



✓ 수집기가 할 일

400·500줄씩 나눠 받기 · 빈 답이면 다시 요청 · **덤**
x월 단위 파일로 저장(작은 파일 문제, 16쪽) · 매일
자동 실행 · 호출 기록 남기기

x 수집기가 안 해도 되는 일

데이터 정리·계산. 받은 그대로 저장하고, 고치는
일은 창고 안에서 Spark가 합니다. 그래야 원본이
남고 "하둡으로 처리했다"가 됩니다.

🔑 인증키는 사람마다 다릅니다. 매뉴얼에 **키 받는 법**을 적고, 받아 둔 **데이터 스냅샷**을 함께 내야 다른 사람도 똑같이
재현할 수 있습니다. API 자체는 기관이 만든 것이라 우리가 만들 필요는 없습니다.

하둡 도구가 뭔가요?

수업 1주차 내용을 그림으로 옮겼습니다.



더 큰 소 한 마리 키우기

더 빠른 CPU, 더 많은 메모리의 큰 컴퓨터 한 대.
데이터가 커지면 따라잡지 못합니다.

한계



소를 여러 마리 쓰기 = 하둡

보통 컴퓨터 여러 대를 묶어 한 대처럼 씁니다.
데이터를 나눠 두고, 있는 곳에서 계산합니다.

하둡의 방식

수업 핵심 문장: “Bring the program to the data rather than the data to the program” —
데이터를 프로그램 쪽으로 옮기지 말고, 프로그램을 데이터가 있는 곳으로 보낸다. (1주차, 그레이스 호퍼의 소 비유)

하둡은 도구 하나가 아니라 “도구 모음”입니다

① 모으기

Sqoop (DB 표) · **Flume** (계속 생기는 파일) · **Kafka**

② 저장

HDFS (분산 창고) · **HBase**

③ 자원 배분

YARN (누가 메모리·CPU를 쓸지 정함)

④ 처리·분석

Spark (계산 엔진) · **Hive** · **Impala** (표·SQL) · MapReduce · Pig

⑤ 화면·관리

Hue (웹 화면) · **Oozie**(작업 예약) · **Sentry**(보안)

파란 글씨 = 우리가 쓸 도구 · 회색 = 이번에는 안 씀. 수업 1주차 요약(p.27)의 분류를 따랐습니다.



수업 진도와 맞춰 보면

4장 Sqoop · 5~6장 Hive/Impala · 7장 파일 형식 · 8장 파티션 · 9장 Flume · 10~17장 Spark. 지금은 6장까지 왔고, 형식·파티션 · Flume · Spark는 이제 배웁니다.



우리 노트북에서는?

실습 VM은 보통 컴퓨터 1대짜리 클러스터입니다. 도구와 명령은 진짜 클러스터와 같고 규모만 작습니다. 컴퓨터가 1대라면 복본도 1벌입니다(VM 확인 뒤 확정).

도구 하나씩 ① 모으고 저장하는 도구



HDFS Hadoop Distributed File System · 2~3장 ✓ 핵심

여러 컴퓨터에 나눠 보관하는 거대한 창고. 큰 파일을 128MB 조각(블록)으로 잘라 흩어 두고, 조각마다 기본 3벌씩 복사합니다.

잘하는 것

아주 큰 파일을 처음부터 끝까지 읽기. 컴퓨터 한 대가 고장
나도 복본으로 버텨

약한 것

작은 파일이 아주 많으면 비효율(파일당 100MB 이상이 좋음).
한 번 쓰면 중간 수정 불가

우리는 이렇게

원본 JSON과 정리본을 보관. 수업 권장 폴더 규칙대로 작업 중은 `/etl`, 완성본은 `/data`. 파일은 `년월` 단위로 크게 묶기



YARN Yet Another Resource Negotiator · 3장 ✓ 자동

메모리·CPU 배차원. 여러 작업이 컴퓨터들의 자원을 나눠 쓰도록 정하고, 데이터가 있는 컴퓨터에서 계산이 돌게 합니다.

잘하는 것

Spark·Hive 같은 여러 도구가 한 클러스터를 함께 쓰게 함.
작업 상태·로그를 한 화면에서 확인

약한 것

직접 다룰 일은 적음. 메모리가 모자라면 작업이 기다리거나
실패

우리는 이렇게

따로 설정할 일은 거의 없음. 작업이 멈추면 YARN 화면에서 원인 확인 → 최종발표의 “디버깅 과정” 증거로 캡처



Sqoop SQL-to-Hadoop · 4장(배움) ✓ 사용

일반 DB와 HDFS 사이의 이삿짐 트럭. MySQL 같은 DB의 표를 가져오고(import), 결과를 DB로 보냅니다(export).

잘하는 것

표 통째로, 일부 열·행만, 새로 생긴 것만(incremental)
가져오기. 기본 4갈래로 나눠 빠르게

약한 것

출발지나 도착지가 DB여야 함 → API는 못 부름. export할
DB 표는 미리 만들어 뒀어야 함

우리는 이렇게

① 덤 좌표·관측소 연결표를 MySQL → Hive로 import ② 분석 결과를 HDFS → MySQL로 export해 웹이 읽게 함 (수업
예제와 같은 방식)



Flume ‘통나무 띄워 보내는 수로’ · 9장(배울 예정) ✓ 사용

계속 생기는 데이터를 HDFS로 흘려보내는 수도관. 여러 곳에서 나오는 로그 같은 이벤트 데이터에 맞습니다.

잘하는 것

매일·매시간 새로 생기는 파일을 자동으로 계속 옮김

약한 것

데이터를 가지러 가지는 않음 → 수집기가 파일을 떨궈 줘야
함. 설정 파일 작성 필요

우리는 이렇게

수집기가 파일을 떨구는 폴더를 지켜보다가 HDFS 원본 폴더로 옮김. 9장 전에는 `hdfs dfs -put` (2주차)으로 손으로 올려도
됨

Kafka — 대량의 실시간 메시지 통로. 우리는 1시간 단위로 불필요

HBase — 한 건씩 빠르게 읽고 쓰는 하둡 DB. 우리는 통째로
분석하므로 불필요

도구 하나씩 ② 표로 보고 계산하는 도구



Hive HiveQL · 5~6장(배움) ✓ 사용

HDFS 파일에 "표 이름표"를 붙여 SQL로 다루게 하는 도구. 표 정보는 메타스토어에 저장되고, 질의는 MapReduce 작업으로 돌아갑니다.

잘하는 것

표 만들기과 대량 정리(ETL). Impala와 같은 표를 공유.
map-array 같은 복합 형식

약한 것

일괄 작업이라 대화하듯 빠른 조회에는 안 맞음. 수업도
"즉석 분석은 Impala"라고 함

우리는 이렇게

원본 폴더에 **EXTERNAL** 표를 걸어 표를 지워도 원본은 남게 함. 원본 표는 모든 열을 글자(STRING)로 두기(16쪽 함정)



Impala 5~6장(배움) ✓ 사용

같은 표를 아주 빠르게 조회하는 전용 SQL 엔진. MapReduce를 쓰지 않아 밀리초~초 단위로 답합니다.

잘하는 것

탐색, 대시보드용 조회, 같은 질문 반복. Hue에서 바로 실행

약한 것

Hive-HDFS에서 바꾼 내용을 바로 모름 → REFRESH 필요.
복합 형식 미지원

우리는 이렇게

데이터 살펴보기, 파티션 전후 속도 측정, 웹에 보낼 결과 확인. 새 데이터를 넣은 뒤엔 꼭 REFRESH



Spark 10~17장(배움 예정) · 수업 18장 중 8장 ✓ 핵심

여러 컴퓨터의 메모리에서 나눠 계산하는 범용 엔진. 이 수업의 데이터 처리 도구입니다.

잘하는 것

여러 단계 계산(정리 → 사건 찾기 → 특징 만들기). 같은
데이터를 메모리에 올려 반복 사용(persist). SQL과 코드 둘
다

약한 것

배우는 데 시간이 톨. 메모리를 많이 써서 노트북 사양
이야기의 주범

우리는 이렇게

원본 → 정리본 변환, 반응 시간 계산, 예측용 학습표 만들기. 모델 학습은 과제가 허용한 Python으로 (18쪽)



Hue Hadoop User Experience · 수업 전반 ✓ 사용

하둡을 웹 브라우저로 쓰는 화면. 명령어 대신 클릭으로 파일 보기·질의·작업 확인을 합니다.

잘하는 것

HDFS 파일 올리기·보기, Hive-Impala 질의, 작업 상태 확인

약한 것

매일 반복하는 자동화에는 안 맞음 → 스크립트로

우리는 이렇게

개발 중 확인용, 발표 영상의 시연 화면

MapReduce — Spark 이전의 계산 엔진. Hive가 속에서 씀

Pig — 스크립트 언어. 이 과목에서 깊게 안 다룸

Oozie △ 선택 — 작업 순서를 정해 예약 실행. 매일 수집→적재→
정리를 묶을 때 고려

Sentry — 접근 권한(보안). 수업 VM에선 불필요

비슷한 도구, 언제 무엇을 쓰나

채점 기준 “왜 그 도구를 골랐나”에 답하는 쪽입니다.

Scoop ↔ Flume — 데이터가 어디서 오나?

Scoop

출발지가 DB의 표일 때

VS

Flume

계속 새로 생기는 파일·로그일 때

→ 우리: API에서 받은 파일은 Flume, 댐 좌표표와 분석 결과는 Scoop

Hive ↔ Impala — 같은 표, 다른 엔진

Hive

큰 일괄 정리·표 만들기(ETL). 느려도 튼튼하게

VS

Impala

빠른 조회·탐색·대시보드. 즉석 분석

→ 우리: 표 정의·정리는 Hive, 조회와 속도 측정은 Impala (메타스토어를 공유하므로 같은 표)

SQL(Hive-Impala) ↔ Spark — 계산이 얼마나 복잡한가?

SQL

고르고, 세고, 합치는 일. 예: 댐별 월평균

VS

Spark

여러 단계·시간 순서 계산. 예: 사건 자르기, 3시간 전 강우 열 만들기

→ 우리: 단순 집계는 Impala, 반응 시간과 예측용 학습표는 Spark

파일 형식(7장) · 파티션(8장) — 어떻게 담을까? 배울 예정

텍스트(JSON·CSV)

사람이 읽기 쉬움. 받은 그대로

VS

Parquet

열 단위 압축. 필요한 열만 읽어 빠름

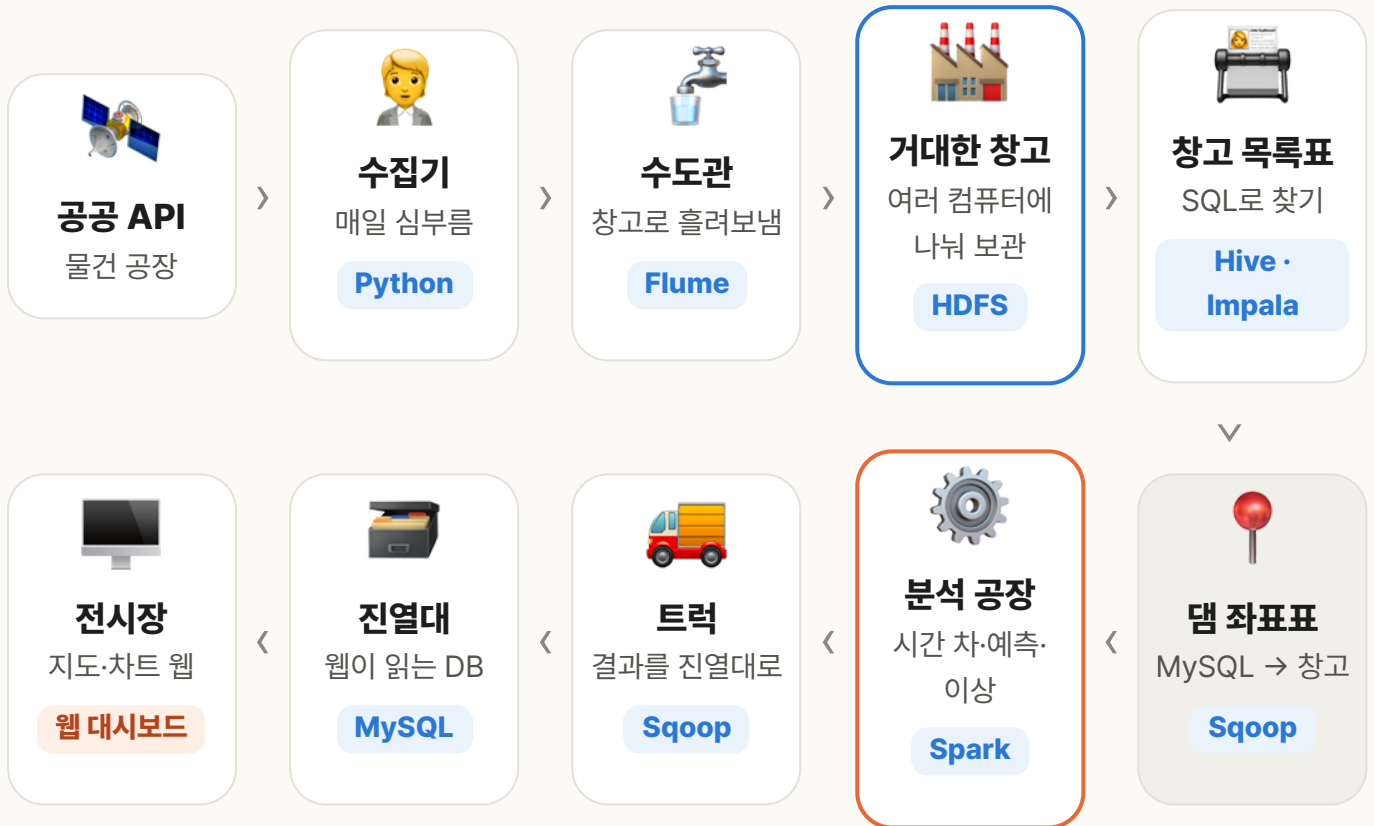
→ 우리: 원본은 JSON, 정리본은 Parquet. 폴더는 연·월로 나눔(댐×날짜처럼 너무 잘게 나누면 작은 파일 문제)

우리의 도구 조합

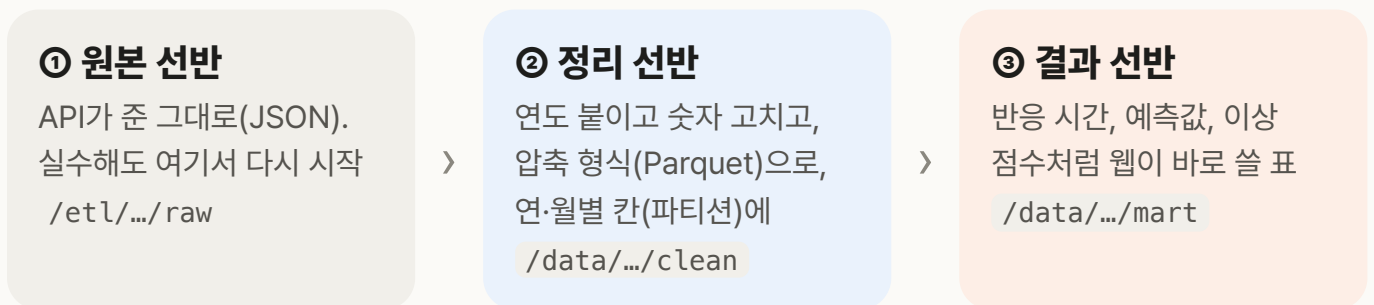
단계	도구	고른 이유
모으기	Python 수집기 → Flume	API는 하둡 도구로 못 부름 → 파일로 받아 Flume이 창고로
기준표	MySQL → Sqoop import	좌표·관측소 연결표는 작은 표라 DB에서 관리
저장	HDFS (/etl → /data)	원본·정리본·결과를 나눠 보관 (수업 폴더 규칙)
표·정리	Hive (EXTERNAL · Parquet · 연월 파티션)	표 정의와 대량 정리. 지워도 원본 보존
조회	Impala + Hue	빠른 탐색과 속도 측정, 시연 화면
계산	Spark (+ Python)	반응 시간·학습표는 Spark, 모델은 Python 허용
보여 주기	Sqoop export → MySQL → 웹	웹이 읽는 진열대. 수업 export 예제와 같은 구조

데이터를 받은 뒤의 전체 흐름

택배 물류센터를 떠올리면 쉽습니다. 파란 글씨가 수업 도구입니다. 도구마다 실제로 하는 일은 다음 쪽에 있습니다.



🏭 창고 안은 선반 3칸으로 나눕니다



🔍 왜 칸을 나누나 (파티션 · 8장에서 배움)

“2023년 7월만 보여 줘” 하면 그 칸만 열면 됩니다. 창고 전체를 뒤지지 않아 빨라집니다.

🗄️ 왜 압축 형식인가 (Parquet · 7장에서 배움)

같은 데이터가 훨씬 작아지고, 필요한 열만 읽어서 빨라집니다. 전후 용량을 재서 발표해 씁니다.

도구별로 실제로 하는 일

넣는 것 → 하는 일 → 나오는 것. 예시 명령은 모양만 보여 주는 것입니다.

도구	넣는 것	하는 일	나오는 것 · 예시
① 🤖 수집기	API 주소, 인증키	댐·관측소별로 나눠 받기, 빈 답 재시도, 댐×월 파일로 저장. 매일 자동 실행	<code>spool/dam_hourly/1012110_2023-07.json</code>
② 🚰 Flume	수집 폴더의 새 파일	폴더를 지켜보다 새 파일을 참고로 옮김 (9장 전에는 손으로 put)	<code>hdfs dfs -put ... /etl/waterpulse/raw/dam_hourly/</code>
③ 🏠 HDFS	원본 JSON	블록으로 나눠 보관. 원본 · 정리본 · 결과 선반으로 나눔	<code>/etl/.../raw → /data/.../clean → /data/.../mart</code>
④ 📁 Hive	HDFS 폴더	폴더에 표 이름표 붙이기. 원본은 EXTERNAL·전부 STRING, 정리본은 숫자·날짜 형식	<code>CREATE EXTERNAL TABLE raw_dam_hourly (obsrdt STRING, inflowqy STRING, ...) LOCATION '/etl/waterpulse/raw/dam_hourly';</code>
⑤ ⚙️ Spark	원본 표	연도 붙이기, 24시 → 0시, 쉼표 빼기, 시험 관측소 빼기 → 정리본 저장. 이어서 반응 시간·학습표 계산	<code>/data/waterpulse/clean/dam_hourly/year=2023/month=7/ (Parquet)</code>
⑥ ⚡ Impala	정리본·결과 표	새 데이터 반영 후 빠른 조회. 파티션 전후 속도 측정	<code>REFRESH dam_hourly; SELECT ... WHERE year=2023 AND month=7;</code>
⑦ 🚚 Sqoop	MySQL 좌표표 / 결과 폴더	좌표표를 참고로 import, 결과를 MySQL로 export	<code>sqoop export --export-dir /data/waterpulse/mart/forecast --table forecast --update-mode alllowinsert ...</code>
⑧ 🖥️ 웹	MySQL 결과 표	지도·차트로 보여 주기	대시보드 (6쪽 화면)

📦 하둡에 넣을 때 조심할 것 3가지 수업에서 배운 내용이 우리 데이터에 그대로 걸립니다

① 작은 파일이 너무 많음

HDFS는 “적당한 수의 큰 파일”에 최적(2주차 p.6). 댐×하루로 받으면 댐 자료만 약 45만 개 → 댐×월로 묶고, Spark로 합쳐 Parquet으로

② 형식이 틀리면 조용히 빈칸

Hive는 읽을 때 형식을 확인 (schema on read, 5주차 p.29). "1,946.781" 이 숫자 열에선 NULL로 사라짐 → 원본 표는 전부 **STRING**, 숫자 변환은 Spark에서

③ Impala는 바뀐 걸 바로 모름

Hive나 HDFS로 넣은 뒤엔 REFRESH 표 (새 데이터), INVALIDATE METADATA (새 표)를 실행 (5주차 p.47~48)

단계별로 하면 이렇게합니다

제안 일정입니다. 팀과 맞춰 조정하세요.



1. 모으기 10/6 ~ 10/11

API → 파일 → Flume → 창고(원본 선반). 오른쪽 합정을 여기서 처리



2. 정리하기 10/8 ~ 10/16

Hive 표 3칸 만들기, Parquet·연월 칸 나누기(7·8장 내용을 미리 적용), 용량·속도 전후 비교 재기



3. 중간발표 10/19

샘플로 만든 표 1개, 반응 시간 차트 1장, 예측 숫자 1개, 구조 그림, 계획



4. 분석 10/20 ~ 11/16

시간 차 → 예측 → 이상 찾기 → (여유 있으면) 묶기



5. 웹 만들기 11/9 ~ 11/30

지도 + 댐 카드 + 차트. Sqoop으로 결과를 MySQL에 내보내 연결



6. 마무리 11/30 ~ 12/7

실습 VM에서 재현 매뉴얼 확인, 데이터 스냅샷 동봉(키는 사람마다 다름), 영상 녹화



오늘 직접 찾은 데이터 합정 6가지

모르고 지나가면 데이터가 조용히 들어옵니다.

- 1 시각에 연도가 없음 07-14 01시
→ 조회한 기간에서 연도를 붙이기
- 2 하루의 끝이 "24시" 07-14 24시
→ 다음 날 0시로 바꾸기
- 3 숫자에 쉼표가 섞임 "1,946.781"
→ 쉼표 빼고 숫자로 바꾸기
- 4 들어온 물이 0 이하 (계산값이라서)
2023년 7월 소양강댐 743시간 중 27시간
→ 표시해 두고 따로 처리
- 5 관측소 목록에 시험용 "시험2"가 섞임
→ 목록에서 빼기
- 6 한 번에 너무 많이 달라면 오류 없이 빈 답
→ 400줄씩 받고, 빈 답은 다시 요청

+ 댐마다 자료가 있는 기간이 다릅니다. 빈 기간을 "비가 안 온 것"으로 착각하지 않기.

1주차에 꼭: 수업 실습 VM(Cloudera 기반)의 Spark·Python 버전을 먼저 확인하세요. 버전에 따라 쓸 수 있는 분석 기능이 달라집니다.

분석은 이렇게 합니다 — 큰 그림

과제가 요구하는 분석과, 수업에서 배우는 도구를 맞춰 봤습니다.

📖 과제가 원하는 분석 3가지 *"trend detection, predictive modeling, comparative analysis"*



추세 찾기

20년 동안 큰비 사건 수, 댐 반응 시간, 최대 유입량이 어떻게 변했나

제안서에 없던 것 · 추가 제안



예측 모델

3·6·12시간 뒤 댐 유입량
+ 댐: 예측과 크게 다른 순간
= 이상 탐지



비교 분석

댐마다 반응 시간이 왜 다른가
(유역 넓이·댐 크기와 비교)

🏗 어떤 분석이든 4단계



① 둘러보기

기간·빈칸·이상값
확인 → "데이터
품질표"



② 사건 자르기

큰비 구간만 골라
사건 목록 만들기



③ 계산·학습

반응 시간 계산,
예측 모델 학습



④ 검증·한계

기준과 비교, 안
맞는 경우 설명

⚙ 수업에서 배울 Spark가 하는 일 10~17장 배울 예정



Spark SQL·DataFrame (17장) — 표 고르기·합치기·시간 밀기. "3시간 전 강우" 같은 열을 만들어 예측용 학습표를 만듭니다.



Pair RDD 집계 (12장) — "댐별로" 묶어 세고 평균 내기. 댐별 사건 수, 반응 시간 가운데 값.



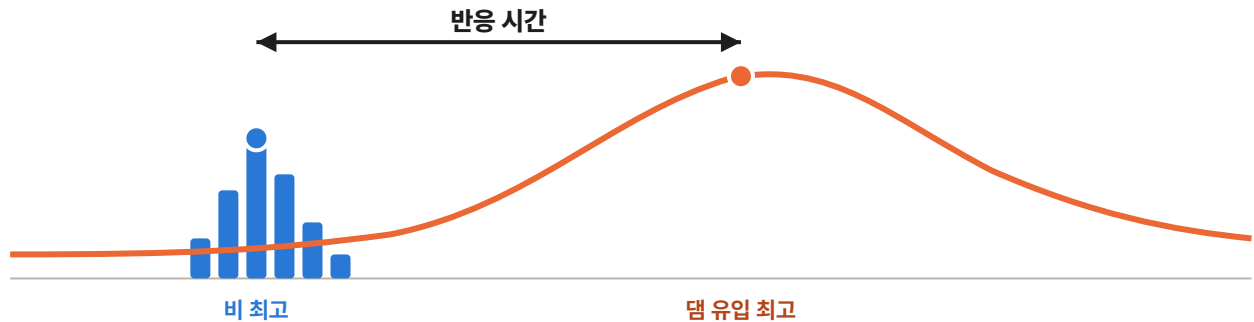
영속화 persist (15장) — 같은 정리본을 여러 번 쓸 때 메모리에 올려 두기. 예측 실험을 반복할 때 빨라집니다.



Python (과제가 허용) — *"any kind of analysis tool or programming languages (e.g., Python)"*. Spark가 줄여 준 표로 그래프와 예측 모델(scikit-learn). 수업 목차에 기계학습 장은 없습니다.

▼ 큰 데이터는 Spark가 줄이고, 작아진 표로 Python이 마무리합니다: 최대 약 1억 줄(HDFS) → Spark → 학습표 수백만 줄 · 사건표 수만 줄 → Python 모델·그래프 → 결과 표 → 웹

분석 ① 시간 차 재기 (비교) + 추세



1 큰비 "사건" 자르기 Spark 시간 순서 계산

유역 평균 비가 기준(예: 6시간에 20mm 이상)을 넘은 구간을 사건 하나로. 기준은 몇 가지로 바꿔 보며 정합니다.

2 사건마다 두 시각 찾기

비가 가장 많이 온 시각, 그 뒤 48시간 안에 댐 유입량이 가장 많은 시각.

3 빼기 = 반응 시간

5쪽 사례: 비 최고 7/14 03시 → 댐 유입 최고 7/14 17시 = **14시간** (5쪽의 비→강 12시간 + 강→댐 2시간).

4 댐별로 모으기 Pair RDD · groupBy

사건 수백 개의 가운데 값(중앙값)과 범위. 평균은 튀는 사건 하나에 휘둘리므로 가운데 값을 씁니다.

5 이유 찾기 = 비교 분석

수문 제원 API의 유역 넓이·댐 높이와 나란히 점그래프로. "유역이 넓을수록 느린가?"

6 추세 = 20년 변화

연도별 큰비 사건 수, 반응 시간, 최대 유입량을 2006 → 2025로 이어 그리기. 연도별 묶기 한 번이면 됩니다.

결과로 나오는 "사건표" (한 줄 = 사건 하나)

댐	비 최고 시각	댐 유입 최고 시각	반응 시간	최대 유입량	유역 넓이
소양강	2023-07-14 03시	2023-07-14 17시	14시간	1,033 m³/s	제원 API
...	댐 59곳 × 20년의 큰비 사건이 한 줄씩 쌓입니다				

💡 다른 방법도 있습니다: 비 그래프를 몇 시간 뒤로 밀었을 때 유입량 그래프와 가장 닮는지 찾기(교차상관). 두 방법의 답을 비교하면 결과를 더 믿을 수 있습니다.

분석 ② 미리 맞히기 + ③ 이상한 순간 찾기

1 학습용 표 만들기 — 한 줄 = “한 댐의 한 시각” Spark DataFrame

댐	시각	지난 1시간 비	지난 3시간 비	지난 6시간 비	상류 강 유량	지금 유입량	저수율	월	정답: 3시간 뒤 유입량
소양강	2023-07-14 12시	0.5mm	1.5mm	8.2mm	328 m ³ /s	770 m ³ /s	59.4%	7	847m³/s

↑ 실제 값입니다(5쪽과 같은 자료). 시간을 앞뒤로 밀어 “지난 몇 시간”과 “몇 시간 뒤” 열을 만듭니다. 이런 줄이 수백만 개 생깁니다.

2 시간으로 나누기

2006~2022년으로 연습(학습), 2023~2025년으로 시험. 섞지 않고 시간으로 자르는 이유: 미래를 맞히는지 보려고.

3 기준선 먼저

“3시간 뒤에도 지금 양 그대로”라고 찍는 방법. 아무것도 안 배운 예측입니다. 모델은 이것보다 나아야 의미가 있습니다.

4 모델은 쉬운 것부터 Python scikit-learn

선형회귀 → 랜덤포레스트·그래디언트 부스팅. 3·6·12시간 각각 따로 학습.

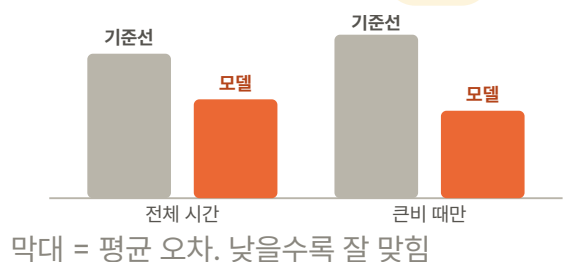
5 채점

평균 오차(평균 몇 m³/s 틀렸나), 기준선보다 몇 % 줄었나. 평소엔 기준선도 잘 맞으므로 **큰비 때만 따로 채점**.

③ 이상한 순간 찾기

- ① 오차 = 실제 - 예측을 매시간 계산
- ② 평소 오차 범위(예: 큰 쪽 1%)를 넘으면 “평소와 다름” 표시
- ③ 먼저 데이터 오류(0 이하, 빈칸, 관측소 고장)를 걸러 따로 표시 → 진짜 변화와 구분

이렇게 보여 줍니다 가상



결과를 믿을 수 있게: 한계와 예외

최종 채점 "분석의 깊이"에 "Consideration of limitations and exceptional cases"가 들어 있습니다.



큰 홍수는 드뭅니다. 20년에 몇 번뿐이라 그때의 성능은 흔들립니다. → 사건 수를 함께 보고, 큰비 때 성능을 따로 채점



유입량은 잔 값이 아니라 계산값입니다. 저수량 변화로 계산해 0 이하나 튀는 값이 섞입니다. → 처리 방법을 밝히고 처리 전후 결과 비교



방류는 사람이 정합니다(댐 운영). 날씨만으로는 못 맞힙니다. → 방류량은 예측 대상이 아니라 입력으로만



관측소는 고장·빈칸이 있습니다. → 댐·관측소별 빈칸 비율표, 채운 방법 기록



댐마다 자료 기간이 다릅니다. → 댐끼리 비교할 땐 같은 기간으로 맞추기



함께 움직인다고 원인은 아닙니다. "유역이 넓으면 느리다"는 경향이지 증명이 아닙니다. → "관계가 보인다"로 표현

🎤 중간발표(10/19)에 보여 줄 분석 최소 세트

✓ 데이터 품질표

샘플 댐별 기간, 빈칸 비율, 0 이하 유입량 비율. 예외를 이미 보고 있다는 증거

✓ 반응 시간 미리보기

댐 3~5곳, 큰비 사건 몇 개로 만든 차트 1장

✓ 예측 숫자 1개

댐 1곳에서 기준선 vs 간단한 모델의 평균 오차

✓ 확장 계획

전체 댐, 추세, 이상 탐지, 웹 화면까지 12/7 일정

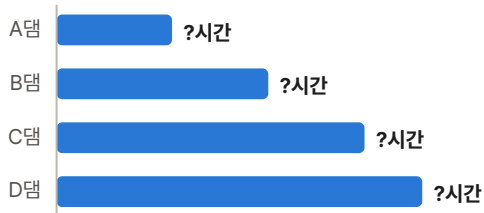
💬 중간발표 채점은 "아이디어·실현 가능성·완성도(예외)·계획"입니다. 분석은 완성본이 아니라 방향과 첫 숫자를 보여 주면 충분합니다.

이런 결과를 기대합니다

⚠ 아직 분석 전이라 숫자는 없습니다. 아래 그림은 결과가 어떤 모양일지 보여 주는 가상 그림입니다.

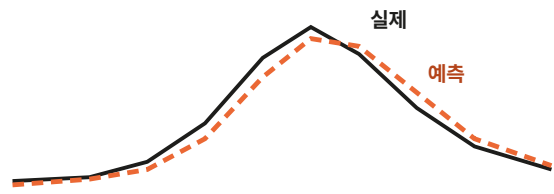
① 댐별 "반응 시간" 순위와 이유 + 추세

비가 온 뒤 몇 시간 만에 물이 몰려오나. 유역이 넓을수록 느린가? 20년 동안 어떻게 변했나?



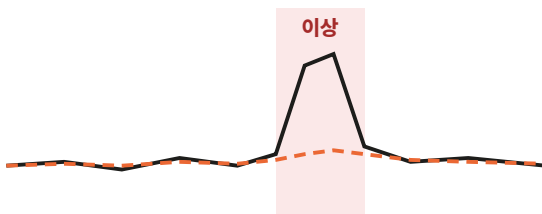
② 3·6·12시간 뒤 예측 vs 실제

"지금 양이 그대로 간다"고 찍는 것보다 몇 % 더 잘 맞히나. 큰비 때는 따로 채점



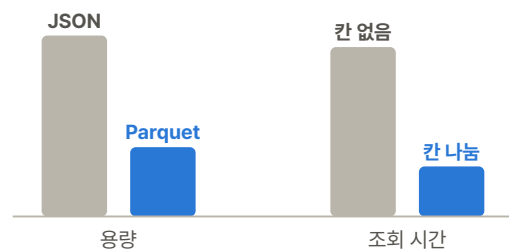
③ 평소와 다른 순간 자동 표시

예측과 실체가 크게 벌어진 때를 빨간 구간으로. 오류값인지 진짜 변화인지도 구분



④ 창고 정리 효과를 숫자로

같은 데이터의 용량, 같은 질문의 조회 시간 전후 비교 (측정 예정)



⑤ 최종: 지도에서 댐을 누르면 지금 상태 · 몇 시간 뒤 예측 · 평소와 다른지 한눈에

6쪽 예시 화면이 실제 데이터로 움직이는 웹 서비스가 최종 목표입니다.

정리하면

1

과제가 원하는 것

공공 데이터 → 하둡 창고에 정리 → 쓸모 있는 분석 → 웹으로 보여 주기. 관리와 분석이 똑같이 중요합니다. 중간발표 10/19(10%), 최종발표 12/7(20%).

2

WaterPulse

비 → 강 → 댐으로 물이 내려오는 “시간 차”를 재고, 몇 시간 뒤 유입량을 미리 맞히는 주제입니다. 보완 4가지: 상류 흐름을 간판으로 · Flume/Sqoop을 넣고 효율은 숫자로 · 분석은 순서대로 깊게 · 데이터 함정을 예외 처리로 발표.

3

API와 수집

5개(수문 운영 정보, 댐코드, 우량수위 관측소, 수문 제원, 기상청 ASOS) 모두 10/5 신청·승인·호출 확인 끝. 샘플은 약 2일, 전체는 약 5일. 수집기도 과제의 일부이며, 데이터 얻는 법을 매뉴얼에 넣어야 합니다.

4

하둡 도구

수집기 → Flume → HDFS → Hive(표) · Impala(빠른 조회) · Spark(계산) → Sqoop → MySQL → 웹. 작은 파일, 형식 오류 NULL, Impala REFRESH를 조심합니다.

5

분석

추세(20년 변화) · 예측(3·6·12시간 뒤, 기준선보다 나은가) · 비교(댐별 반응 시간과 이유) + 이상 탐지. 큰 데이터는 Spark가 줄이고 모델은 Python. 한계와 예외를 함께 발표합니다.

이번 주에 할 일 3가지



수집 시작

전체를 받는 데 약 5일. 늦게 시작하면 중간발표 준비 시간이 줄어듭니다.



실습 VM 확인

Spark·Python 버전부터. 최종 결과는 VM에서 재현돼야 합니다.



역할 나누기

수집·Flume / Hive·Sqoop·MySQL / Spark 분석 / 웹. 넷이 겹치지 않습니다.

📁 관련 파일: 과제/WaterPulse/ — API 확인 스크립트(api_check), K-water 기술문서(docs), 실제 호출 샘플(data/sample), 이 문서의 원본(report)